VERA System Documentation

Inner Workings, Architecture & Design Decisions

This document explains HOW VERA thinks — the mechanics behind every intelligence feature, data flows between systems, and the reasoning behind key design decisions. For setup and configuration, see README.md. For how to use the dashboards, see the user guide, the cheat sheet and Victoria's lite guide in this folder.

Table of Contents

1. Design Philosophy
2. The Nightly Intelligence Pipeline
3. Flag System Deep Dive
4. Signal Learning & Noise Filtering
5. Cross-Domain Pattern Recognition
6. Pacing & Capacity Mode
7. Anticipator — Reminder Engine
8. Chat System Architecture
9. Slack Integration Architecture
10. Travel Pipeline
11. PTO & Clear Windows
12. Summaries Architecture
13. Monthly Life Review
14. Configuration Architecture
15. Trip Identity, Emails & Readiness
16. Packing, Luggage & Leaving Home
17. Finances (November 2026)
18. Places, Gifts, Life & Career
19. Family Planning & the Cycle Tracker
20. Dashboards, Navigation & Notifications
21. Appendix: Developer Reference

---

1. Design Philosophy

Why Google Apps Script?

Google Apps Script runs in Google's cloud with zero infrastructure cost and no deployment complexity. The alternative — a hosted server (VPS, Lambda, etc.) — would require managing uptime, auth, cron scheduling, and API proxying. Apps Script eliminates all of that:

- **Native Google service access**: `CalendarApp`, `GmailApp`, `DriveApp`, `SpreadsheetApp` are first-class citizens with pre-authorised OAuth. No token management needed.
- **Built-in time-based triggers**: `ScriptApp.newTrigger('nightlyRun').atHour(23).everyDays(1)` replaces an entire cron + server setup.
- **No cold starts for the nightly batch**: The 11pm trigger fires reliably against Google's own infrastructure.
- **Execution logs**: Every `Logger.log()` call is persisted and viewable in the Apps Script execution log — no log aggregation service required.

The trade-offs are real: 6-minute execution time limit, no persistent process, no in-memory caching across executions. These are worked around with Script Properties (persistent key-value store) and per-execution caching via `_configCache_`.

Why Google Sheets as the database?

Google Sheets serves as VERA's operational database. This is a deliberate choice, not a workaround:

- **Visible to the user**: Ahmed can open the sheet, see every flag, task, and metric in a grid, and manually edit any row. There is no hidden state.
- **No schema migrations**: Adding a new column to `TASK_HEADERS` requires updating the constant in `Code.js` and ensuring `ensureSheet()` writes the header on the next run. No SQL ALTER TABLE.
- **Tab-as-namespace**: Each domain (Flags, Tasks, Goals, Itinerary, GymLog, etc.) lives in its own tab. The `TABS` object in `Code.js` defines 70+ tab names — each is effectively a table.
- **Formula-driven dashboards**: The React dashboard reads data via the Web App API, but power users can apply Google Sheets formulas directly to any tab for custom analysis.
- **Audit trail by default**: Sheets preserves edit history natively. Every nightly write is visible in the revision history.

The schema for every tab is defined as a constant array in `Code.js` (e.g. `FLAG_HEADERS`, `TASK_HEADERS`, `GOAL_HEADERS`). The `ensureSheet()` function creates tabs with headers on first run and skips if they already exist — making the system safe to re-run `setupVERA()` without clobbering data.

Why Claude AI for flags (not pure rules)?

Rule-based systems excel at precise conditions: "bill due in 5 days" or "task 14+ days old". But life is ambiguous:

- A meeting labeled "Team Sync" might require Ahmed to prepare a deck — or not. Context determines which.
- A task called "follow up with doctor" means something different if there is an upcoming health appointment on the calendar.
- Spotting that a bucket-list destination is adjacent to an upcoming trip requires geographic reasoning, not a simple date comparison.

Claude reasons over the full assembled context (calendar events, open tasks, summaries, PTO status, interest ledger, bills, home items, goals, trips, financial goals) in a single pass and surfaces what matters. The prompt includes explicit rules for each domain (see Section 2 / `buildPrompt()`) but Claude applies them with judgment, not literal pattern matching.

Rule-based vs AI-assisted split

The split is intentional and deliberate:

Approach	Used for
Rule-based	Escalation aging (3d/7d thresholds), pacing mode miss scoring, pattern recognition triggers, reminder cooldowns, flight status polling, PTO calculation, bill due-date detection
AI-assisted (Claude)	Nightly flag generation, chat responses, packing list generation, weekend planning memos, monthly review synthesis, Explorer daily discovery

Rule-based checks run first, every night, regardless of AI availability. If the Claude API call fails, all the rule-based flags and Slack logs still go out. The Claude call is the only step that can produce zero output on a network error — everything else is local computation.

Why no build pipeline?

The React dashboard (`index.html`) uses Babel Standalone loaded from a CDN. This means:

- No `npm install`, no `webpack`, no build step.
- The file is a single HTML file that can be opened locally, pushed to GitHub Pages, or deployed to Netlify as a static site.
- The entire frontend state is managed with React hooks; no Redux, no router.
- Dark VERA theme is pure CSS variables defined in a `<style>` block.

The trade-off is a slightly longer initial parse time for Babel in the browser. For a personal dashboard used by two people, this is acceptable.

---

2. The Nightly Intelligence Pipeline

`nightlyRun()` fires at 11pm daily via the Apps Script time-based trigger installed by `setupTriggers()`. It is the orchestration function: it runs every pre-computation step in order, then calls Claude, then writes flags.

All pre-computation steps (Steps -1 through 0q) are wrapped in individual `try/catch` blocks. Failures are pushed into a `stepFailures` array and reported to `#vera-logs` at the end of the run — they do not abort the pipeline. Steps 1–4 (data collection, Claude call, flag write) are inside a single outer `try/catch`; a failure here sends an error email to `CONFIG.MORNING_NUDGE_EMAIL` and a failure message to `#vera-logs`.

Step -1: `escalateAgedFlags_()`

Scans every unresolved, unacknowledged flag in the `Flags` tab. For each flag:

- If age >= 3 days and `Escalated` column is empty: bumps urgency up one level (Low→Medium, Medium→High, High stays High) and writes `'3d'` to the `Escalated` column (col K).

- If age >= 7 days and `Escalated` column is `'3d'`: appends a stale note to the `Reason` field and writes `'7d'` to `Escalated`.

Snoozed flags are skipped while `Snoozed Until` is a future date. Each threshold fires exactly once per flag — the `Escalated` column tracks state.

Step 0: `writeSummarySnapshot()`

Calls `writeMetrics_(ss)` and `writeSummaries_(ss)` in sequence. Both operations clear `[AUTO]`-prefixed rows from their respective tabs and rewrite them fresh. This runs before Claude is called so the AI sees current data.

- `writeMetrics_()` writes task counts, calendar density, and flag counts to the `Metrics` tab.
- `writeSummaries_()` reads `summary_sheet:SourceName` Config rows and pulls live cell values from external Google Sheets.

This step is non-fatal at the outer level but each individual tab write is wrapped internally.

Step 0a: `syncCalendarBirthdaysToImportantDates_()`

Reads upcoming birthday events from the Joint Chaos calendar (events matching a birthday keyword pattern) and writes them to the `Important Dates` tab if not already present. This keeps the People tab's Important Dates populated automatically without manual entry.

Step 0a-ii: `resetChoresByCadence_()`

Scans the `Chores` tab. For each chore, checks whether enough time has elapsed since `Checked At` based on the `Cadence` column (Daily/Weekly/Bi-weekly/Monthly/Quarterly). If the cadence interval has passed, resets `Checked` to false so the chore appears unchecked in the dashboard for the new period.

Step 0b: `writePTOSnapshot_()` → returns `ptoStats`

Reads Ahmed's work calendar (default: `Verizon Calendar`) and classifies all-day events as vacation days, personal time, or company holidays based on title keywords configured in the Config tab. Computes days used, days remaining, burn-rate vs ideal pace, and upcoming travel windows. Writes `[AUTO]` rows to the `Summaries` tab. Returns a `ptoStats` object that is passed downstream to the Monthly Review (Step 0o) and the Claude prompt to avoid re-reading the PTO data twice.

Step 0c: `runExplorer_()`

Calls Claude in "discovery mode" — a separate prompt (built by `buildExplorerPrompt_()`) that asks Claude to surface 2–3 non-obvious, forward-looking observations connecting goals, tasks, calendar, and the Shared Interest Ledger. Uses a date-keyed entry in `Reminders Memory` to fire at most once per day. Output is delivered via `sendNudge_()` — Slack if configured, email otherwise.

Step 0d: `getSuppressedKeyPatterns_()`

Reads the `SignalLearning` tab and returns all patterns where `Suppressed = Yes`. The returned array is passed into `generateFlags()` (Step 2&3) and injected into the Claude prompt as a "SUPPRESSED TOPICS" list — patterns the user has consistently snoozed or ignored will not be re-flagged. See Section 4 for the full scoring logic.

Step 0e: `recordExpiredFlags_()`

Scans the Flags tab for flags that are open (unresolved, unacknowledged, unsnoozed) and either: (a) older than 30 days, or (b) marked `Escalated = '7d'`. For each, calls `recordFlagOutcome_(flagKey, 'expired')` to add a noise-score penalty in Signal Learning. Uses the `Escalated = '7d'` marker as a proxy for "VERA tried, user never responded."

Step 0f: `checkPreTripBriefings_()`

Reads the `Itinerary` tab, groups rows by trip key (`YYYY-MM-DD|Trip Label` format), and finds trips whose departure date is within the configured `pretrip_briefing_hours` window (default: 48 hours). For each qualifying trip, assembles a structured briefing flag (via `buildPreTripBriefingFlag_()`) covering weather, packing completion, flight status, itinerary items, and confirmation numbers. Deduplication is handled by `writeFlags()` — the flag key ensures one briefing per trip, per lifetime.

Step 0g: `checkPostTripCapture_()`

Fires `posttrip_capture_delay_days` (default: 1) days after a trip's end date. Writes a flag prompting Ahmed to log trip memories, expenses, and notes. Gated by `posttrip_capture_enabled` Config key.

Step 0h: Morning routine reset

Directly reads the `Morning Routine` tab and sets `Checked = false` and `Checked At = ''` for every item row (columns 5–6). This resets the daily checklist so it starts fresh each morning. Intentionally a direct sheet write, not a function call — it is simple enough to not warrant its own module.

Step 0i: `checkGymSessions_()`

Reads the `Gym Log` tab and the calendar within the configured `gym_tracker_lookback_hours` window (default: 24 hours). Looks for calendar events with titles matching gym/workout patterns that have ended. Writes new rows to `Gym Log` when found. Gated by `gym_tracker_enabled = 'true'` Config key.

Step 0j: `checkFitnessConsistency_()` + `checkFitnessTravelGap_()`

`checkFitnessConsistency_()` evaluates gym session count vs the `fitness_weekly_target` Config key. If behind pace by the configured `fitness_low_flag_day` (default Wednesday = day 4), writes a Low-urgency flag. `checkFitnessTravelGap_()` checks if a trip is approaching and automatically creates gym session placeholder events in the VERA calendar for the travel period. Both are gated by `fitness_enabled = 'true'`.

Step 0k: `autoRestockItems_()` + `generatePantryFlags_()`

`autoRestockItems_()` uses exponential moving average (EMA, configurable alpha via `pantry_ema_alpha`, default 0.3) to predict when household items will run out based on `Purchase History` tab data, and pre-adds restock items to the Shopping List. `generatePantryFlags_()` checks if a trip is approaching and items are predicted to run out during the trip. Both are gated by `pantry_enabled = 'true'`.

Step 0l: `inferCapacityMode_()`

Reads tomorrow's calendar and scores load based on meeting count and event density. Writes a capacity mode signal (`busy`, `normal`, or `light`) to Script Properties. The `morningNudge()` function reads this to filter which flags to surface in the morning briefing — `busy` days show only High + time-sensitive flags.

Step 0m: `checkContracts_()`

Reads the `Contracts` tab. For each active contract, checks whether the end date is within the notice period (stored in `Notice Period Days` column). Generates Medium flags for contracts approaching their notice window and High flags for expired contracts. Uses source `'General'` and key format `contract_renewal_[name_snake]`.

Step 0n: `checkHealthAppointments_()`

Reads Google Calendar for events prefixed with `DR:` (the convention for health appointments). Computes days until the appointment and compares against configured intervals. Generates flags for overdue or upcoming appointments. This replaces the former `HEALTH_APPOINTMENTS` sheet tab — appointment data now lives in the calendar, not a sheet.

Step 0o: `checkMonthlyReview_(ptoStats)`

Fires only on the 1st of each month. Checks whether a review for the prior month already exists in the Flags tab (dedup by key `monthly_review_YYYYMM`). If not, calls `buildMonthlyReview_()`, writes a Low-urgency flag, archives to the `Monthly Reviews` tab, and sends a styled HTML email. The `ptoStats` object computed in Step 0b is passed in to avoid re-reading PTO data. See Section 13 for the full assembly.

Step 0q: `checkCrossPatternFlags_()`

Builds a cross-domain signal snapshot and evaluates 7 compound patterns. See Section 5 for the full pattern definitions and trigger conditions. Writes at most `pattern_max_flags` (default: 2) new flags per run.

Step 1: Data collection

```
events    = getUpcomingEvents()       // Calendar.js — reads all Google Calendars
tasks     = getOpenTasks()            // Tasks.js — reads Tasks tab + Google Tasks
summaries = getSummaries()            // Summaries.js — merges Metrics + Summaries tabs
ledger    = getSharedInterestLedger_() // Interests.js — reads Shared Interests tab
```

`getUpcomingEvents()` reads the number of days configured in `calendar_days_ahead` (default: 7). It reads ALL calendars via `CalendarApp.getAllCalendars()`, applying `skip_calendars` Config rows to exclude noise calendars. Each event gets a `calLabel` from `calendar_label:CalName` Config rows.

Step 1b: `suggestDueDates(tasks)`

For tasks with no due date, Claude suggests a due date based on task content and writes it back to the sheet. This is a separate, non-fatal Claude call made before the main flag generation call.

Step 2: Early exit if no data

If `events.length === 0 && tasks.length === 0 && summaries.length === 0`, the Claude call is skipped entirely. A log entry and a Slack message are sent (`0 flags — no data`). This prevents burning API tokens on empty runs (e.g. quiet travel weekends with no tasks).

Steps 2 & 3: `generateFlags()` — Claude API call

`generateFlags()` in `Claude.js` calls `buildPrompt()` to assemble the full context string, then calls the Anthropic API. The `suppressedPatterns` array from Step 0d is passed in and injected as a "SUPPRESSED TOPICS" block in the prompt. The response is expected to be a raw JSON array — `parseFlags()` strips any markdown fences and JSON-parses the result.

Step 4: `writeFlags(flags)`

Persists Claude's output to the `Flags` tab. See Section 3 for the full dedup and fingerprinting logic.

Step 4b: `recordFlagsGenerated_(flagKeys)`

Extracts the `key` field from each written flag and calls `recordFlagsGenerated_()` in `SignalLearning.js`, incrementing the `Total Seen` counter for each pattern. This feeds the noise-scoring loop described in Section 4.

Slack summary: `sendSlackLog_()`

At the end of `nightlyRun()`, a summary is posted to `#vera-logs`:

```
■ Nightly run — 5 flags written (2 High, 2 Med, 1 Low) · 1 step warning in 47s
```

If `stepFailures` is non-empty, a second message lists each failing step. If the outer try/catch catches a fatal error, it sends a `■ Nightly run FAILED` message with the error and file/line.

---

3. Flag System Deep Dive

Flag creation — `writeFlags()`

`writeFlags(flags)` is called from `nightlyRun()`, `checkCrossPatternFlags_()`, `checkPreTripBriefings_()`, `checkMonthlyReview_()`, `checkMissRate_()`, `EmailParser.js`, and other rule-based checkers. It is the single write path for all flags.

Each flag object must contain: `source`, `flag`, `reason`, `urgency`, and optionally `key`.

Fingerprinting and dedup: Before writing, `writeFlags()` builds a `Set` of fingerprints from ALL existing flags (regardless of state — open, acknowledged, snoozed, or resolved). Once a key has been written in any state, it is permanently blocked from re-appearing. If the issue genuinely recurs, Claude is expected to generate a new key (e.g. adding a `_q2` suffix).

The fingerprint is computed by `makeFlagFingerprint_(source, flagText, key)`:

- If a stable `key` is present: fingerprint = `'key:' + safeKey` (key alone, no source prefix)
- If no key (legacy rows): fingerprint = `source + '|' + first 8 words of flag text`

Fuzzy dedup via `keysAreSimilar_()`: For key-bearing flags, checks whether the new key shares >= 60% of its meaningful tokens with any existing key fingerprint in the set. Month names and standalone numbers are stripped before comparison, so `verizon_bill_march_13` and `verizon_bill_march_14` are treated as the same issue. This prevents date-drifted duplicates when a recurring bill flag shifts by one day.

Flag ID format: `FLAG-YYYYMMDD-NN` where `NN` is a random two-digit number (10–99) chosen at write time. The randomness prevents collision when multiple flags are written in the same nightly run.

Color coding: After writing, `colorCodeFlags()` applies background colors to all rows in the Flags tab:

- High = `#fce8e6` (soft red)
- Medium = `#fef9e7` (soft yellow)
- Low = `#e6f4ea` (soft green)

Flag key convention

Keys are stable, snake_case identifiers designed by Claude. The convention is `domain_description_context`, for example:

- `health_appt_ahmed_annual_physical`
- `cross_high_stress_compound`
- `verizon_bill_march_13`
- `task_neglect_TASK_0012`
- `fgoal_emergency_fund_at_risk`
- `packing_not_started_austin_trip`

Keys serve as the cross-run dedup anchor. When Claude generates the same conceptual flag on consecutive nights (e.g. a bill that is still unpaid), the key should match the existing flag in the sheet, causing `writeFlags()` to skip it as a duplicate.

Escalation — `escalateAgedFlags_()`

Escalation runs at the very start of `nightlyRun()` (Step -1) so the urgency of aged flags is correct before the morning nudge reads them. The `Escalated` column (col K, index 10) stores `'3d'` and `'7d'` as watermarks so each threshold fires exactly once per flag lifetime. Snoozed flags are excluded from escalation while the snooze date is in the future.

Flag lifecycle

```
created (Open)
    ↓
acknowledged (Acknowledged = 'Yes') OR
snoozed (Snoozed Until = future date) OR
resolved (Resolved = 'Yes')
```

Snoozed flags re-surface automatically: `writeFlags()` does not block them, and `escalateAgedFlags_()` skips them while snoozed — but the flag is still visible in the sheet and the dashboard. When the snooze date passes, the flag becomes active again.

When a user resolves a flag (via the dashboard or Slack button), `recordFlagOutcome_(flagKey, 'resolved')` is called, adding a positive signal score to the pattern in Signal Learning.

---

4. Signal Learning & Noise Filtering

Signal Learning tracks how Ahmed engages with each flag pattern over time and uses that history to suppress recurring noise.

`SignalLearning` tab structure

Headers: `Key Pattern | Total Seen | Acknowledged | Snoozed | Resolved | Expired/Ignored | Last Seen | Score | Suppressed`

The `Key Pattern` column stores the output of `extractKeyPattern_()`, which strips date and sequence suffixes from full flag keys:

- `verizon_bill_20260315` → `verizon_bill`
- `task_neglect_TASK_0012` → `task_neglect`
- `ergonomic_break_1234` → `ergonomic_break`

This normalization is critical: it means all variants of a recurring pattern accumulate into one row rather than spreading across dozens.

Score formula

```
score = 100 - (snoozed × 20) - (expired × 15) + (ack × 10) + (resolved × 25)
score = clamp(0, 100)
```

- **Snoozed (-20)**: User acknowledged the flag was surfaced but chose to defer — mild negative signal.
- **Expired/Ignored (-15)**: Flag sat open > 30 days with no action — silent dismissal.
- **Acknowledged (+10)**: User read it and marked it done — mild positive signal.
- **Resolved (+25)**: User took action and resolved the underlying issue — strong positive signal.

Suppression threshold

A pattern is marked `Suppressed = Yes` when: `score < 25` AND `total_seen >= 5`.

The minimum sightings gate (5) prevents suppression from triggering on a pattern that was snoozed once or twice early on. The score threshold (25) means a pattern needs to be consistently ignored or snoozed multiple times before VERA stops surfacing it.

How suppression flows into the pipeline

1. **Step 0d** (`getSuppressedKeyPatterns_()`) reads the `SignalLearning` tab at the start of `nightlyRun()` and returns all suppressed patterns.
2. The array is passed into `generateFlags()` → `buildPrompt()` → injected as a "SUPPRESSED TOPICS" block: *"user has consistently indicated these are not useful — do NOT generate flags for them."*
3. **Anticipator rules** (Reminders.js) check `getSuppressedKeyPatterns_()` before sending nudges for bills due, packing reminders, goal check-ins, and home service items.

Feedback loop

- `recordFlagsGenerated_(flagKeys)` is called after each `writeFlags()` call — increments `Total Seen` for each pattern key.
- `recordExpiredFlags_()` is called each night (Step 0e) — increments `Expired/Ignored` for flags open > 30 days.
- `recordFlagOutcome_(flagKey, outcome)` is called when a user acknowledges, snoozes, or resolves a flag via the dashboard or Slack.

`getTopSignalPatterns_(n)` returns the top N patterns by score (highest first). These are injected into the Claude prompt as "SIGNAL LEARNING — what Ahmed engages with most" to bias flag priority toward topics he cares about.

---

5. Cross-Domain Pattern Recognition

`PatternRecognition.js` implements meta-intelligence: compound signals that span multiple VERA data domains. Single-domain flags (one overdue task, one missed gym session) are left to their individual checkers. The Pattern Recognition engine only fires when two or more domains signal simultaneously.

`buildCrossDomainSnapshot_()`

Assembles a lightweight signal object from existing VERA data functions. Every data read is wrapped in its own `try/catch`: if a domain is unavailable (e.g. Gym Log tab empty), that field is set to `null` and any pattern requiring it is skipped gracefully. The snapshot includes:

Field	Source
`overdueTaskCount`	`getOpenTasks()` — tasks where `isOverdue = true`
`neglectedTaskCount`	`getOpenTasks()` — tasks where `ageInDays >= 14`
`calendarEventsThisWeek`	`getUpcomingEvents()` — events in next 7 days

Field	Source
`activeGoals`	`getGoals_()` — goals with status `'Doing'`
`gymSessionsThisWeek`	`getGymLog_()` — sessions attended in last 7 days
`gymTargetPerWeek`	Config key `gym_sessions_per_week` (default 3)
`overdueHealthAppts`	`getHealthAppointments_()` — `daysUntil < 0` and `intervalMonths != 0`
`activeHighFlags`	Direct Flags tab read — unacknowledged, unresolved, urgency = `'High'`
`pacingMode`	`isInPacingMode_()`
`vacationMode`	`isInVacationMode_()`
`intensityLevel`	`computeIntensitySignal_()` — see Section 6
`takeoutRatio`	`getMealPlanHistory_(2)` — takeout/eating-out fraction of meals in past 7 days

`evaluateCrossPatterns_(snap)` — the 7 patterns

All patterns null-check their required fields before evaluating. A pattern whose required fields are null is silently skipped.

Pattern 1: High-Stress Compound (`cross_high_stress_compound`, urgency: High)

- Trigger: `overdueTaskCount >= 3` AND `activeHighFlags >= 2` AND `gymSessionsThisWeek === 0`
- Rationale: tasks are piling up, high-urgency flags are unresolved, and the user isn't getting physical release — a compounding stress signal.

Pattern 2: Goal–Behaviour Drift (`cross_goal_behaviour_drift`, urgency: Medium)

- Trigger: `activeGoals >= 2` AND `gymSessionsThisWeek < gymTargetPerWeek * 0.5` AND `overdueTaskCount >= 2`
- Rationale: stated goals are active but execution is stalling on both fitness and tasks — declared intent is diverging from actual behaviour.

Pattern 3: Social/Calendar Gap (`cross_social_calendar_gap`, urgency: Low)

- Trigger: `calendarEventsThisWeek <= 1` AND `pacingMode === false` AND `vacationMode === false`
- Rationale: an unusually empty week that is not intentional (not pacing mode, not vacation) — may indicate social drift or missed opportunity for connection.

Pattern 4: Overload + Pacing Mismatch (`cross_overload_no_pacing`, urgency: Medium)

- Trigger: `intensityLevel === 'high'` AND `pacingMode === false` AND `activeHighFlags >= 2` AND `overdueTaskCount >= 3`
- Rationale: the system signals high load but the user hasn't activated pacing mode — they are running hard without a buffer.

Pattern 5: Meal Chaos (`cross_meal_chaos`, urgency: Low)

- Trigger: `takeoutRatio >= 0.7` AND `overdueTaskCount >= 2`

- Rationale: mostly takeout combined with task pile-up is a compound signal of a chaotic life period, not just a food preference.

Pattern 6: Backlog Accumulation (`cross_backlog_accumulation`, urgency: Medium)

- Trigger: `neglectedTaskCount >= 5` AND `calendarEventsThisWeek >= 3`
- Rationale: the calendar is full but tasks aged 14+ days are building up — the user is attending meetings but not processing their backlog.

Pattern 7: Health Neglect Compound (`cross_health_neglect`, urgency: Medium)

- Trigger: `overdueHealthAppts >= 2` AND `gymSessionsThisWeek === 0`
- Rationale: multiple overdue appointments combined with no gym sessions this week is a compound health-neglect signal.

Deduplication and rate limiting

Candidates are sorted High → Medium → Low. Before writing, the function reads all existing `Pattern Recognition` source flags and builds a `suppressedKeys` map: a pattern key is suppressed if it has an open (unresolved) flag, or if it was resolved within the last `pattern_dedup_days` (default: 7) days. A maximum of `pattern_max_flags` (default: 2) new flags are written per run. Both values are overridable via Config tab keys `pattern_max_flags` and `pattern_dedup_days`.

---

6. Pacing & Capacity Mode

`computeIntensitySignal_(activeFlags, tasks, events)`

Defined in `WeekendPlanner.js` but used by both the Weekend Planner and `PatternRecognition.js`. Evaluates three thresholds:

- Active unresolved flags >= 4 → threshold breach
- Overdue tasks >= 2 → threshold breach
- Meetings in the next 5 days (non-all-day events) >= 8 → threshold breach

`level = 'high'` if 2+ thresholds breached, `'medium'` if 1 threshold breached, `'low'` otherwise.

Returns `{ level, activeFlagCount, overdueTaskCount, meetingCount }`.

`isInPacingMode_()`

Fast read from Script Properties. Reads `PACING_MODE_ACTIVE`. Auto-expires: if `today > PACING_MODE_ENDS` date, sets `PACING_MODE_ACTIVE = 'false'` and returns false. Called from `PatternRecognition.js`, `Fitness.js`, and `Reminders.js`.

`isInVacationMode_()`

Fast read: returns `true` if `VACATION_MODE_ACTIVE === 'true'` in Script Properties. Set by `checkVacationMode_()` which runs inside `checkPacing_()` at the start of each nightly run.

`checkVacationMode_()` — vacation detection logic

Reads the `Itinerary` tab to build a date-range map (`tripKey → { min, max }`). Reads `TripMeta` to build a traveler map (`tripKey → traveler string`). Finds the first active trip where today falls within the date span and the trip is not Victoria-only (a trip is Victoria-only if the traveler string contains `'victoria'` and does not contain `'ahmed'`). Writes `VACATION_MODE_ACTIVE`, `VACATION_MODE_ENDS`, and `VACATION_TRIP_NAME` to Script Properties. Fires a Memory Log event on first detection and on transition back to inactive.

`checkMissRate_()` — the pacing offer loop

Scores 4 domains for misses in the last 48 hours:

1. **Gym**: any scheduled session today or yesterday with `attended != 'Yes'`
2. **Tasks**: any Open task with a due date of today or yesterday
3. **Flags**: `>= pacing_flag_threshold` (default 3) unacknowledged Medium/High flags open
4. **Chores**: any chore whose last `Checked At` + cadence interval has elapsed

If 2+ domains hit:

• **First detection**: Writes a Medium flag offering to activate pacing mode. Stores `PACING_FIRST_DETECTED` (timestamp) and `PACING_OFFER_FLAG_KEY` in Script Properties.

• **48 hours later (still 2+ domains hit, offer flag unacknowledged)**: Auto-activates pacing mode. Defers all upcoming non-recurring open tasks with a future due date to next Monday. Sets `PACING_MODE_ACTIVE = 'true'` and `PACING_MODE_ENDS = today + pacing_mode_days` (default 7 days). Writes a Low flag confirming activation.

Pacing mode effects: Pattern 4 (Overload+Pacing Mismatch) and Pattern 1 (High-Stress Compound) suppress their flags while pacing mode is active. The Anticipator's hydration, ergonomic, and mobility rules still fire — only contextually inappropriate nudges are suppressed.

---

7. Anticipator — Reminder Engine

`hourlyCheck()` fires every hour via the Apps Script `everyHours(1)` trigger installed by `setupTriggers()`. Apps Script does not support conditional hour windows in trigger configuration — the function itself contains all guard logic.

`runAnticipatorRules_(now, hour, isWeekday, cfg)`

Runs 8 rules in sequence. Each rule is wrapped in its own `try/catch` — a failure in one rule does not block the others.

Rule	When it fires	Cooldown
`checkErgonomicBreak_`	Weekdays 9am–6pm	55 min (Slack) / 180 min (email)

Rule	When it fires	Cooldown
`checkHydration_`	Weekdays 8am–6pm	110 min (Slack) / 180 min (email)
`checkCalendarOpportunity_`	Weekdays 9am–5pm	90 min (Slack) / 180 min (email), once per date+hour key
`checkEveningMobility_`	Configured hour (default 8pm)	1440 min (once per day)
`checkBillsDue_`	Daily 8–9am	1440 min per bill per day
`checkTripPackingReminder_`	Daily 8–9am	1440 min per trip per day
`checkGoalCheckin_`	Mondays at `weekend_planner_hour` (default 8am)	10080 min (once per week)
`checkHomeServiceDue_`	Daily 8–9am	1440 min per item per day

`Reminders Memory` tab (`TABS.REMINDERS_MEMORY`)

Every sent nudge is logged to this tab with columns: `Rule Key | Sent At | Message` (first 100 chars). `wasRecentlySent_(ruleKey, cooldownMinutes)` scans from the bottom (most recent first) for the most recent entry matching the rule key. If the elapsed time since that entry is less than `cooldownMinutes`, the rule is blocked.

Suppression integration

`checkBillsDue_()`, `checkTripPackingReminder_()`, `checkGoalCheckin_()`, and `checkHomeServiceDue_()` each call `getSuppressedKeyPatterns_()` before firing. If the pattern key for that bill, trip, or home item is suppressed, the nudge is skipped entirely.

`sendNudge_(ruleKey, subject, message)`

Delivery logic:

1. If `isSlackConfigured_()` is true (i.e. `SLACK_BOT_TOKEN` is set in Script Properties): sends to `#vera-notifications`.
2. Else: sends a plain-text email via `MailApp.sendEmail()` to `CONFIG.MORNING_NUDGE_EMAIL`.

Always calls `markSent_(ruleKey, message)` after delivery to record the send in `Reminders Memory`. Also calls `sendSlackLog_()` to post a delivery confirmation to `#vera-logs`.

Weekend Planner integration

`hourlyCheck()` contains an additional gate: `if (day === 1 && hour === cfg['weekend_planner_hour'])`, which calls `runWeekendPlanner_()`. This fires every Monday at the configured hour (default 8am). The planner uses a 9000-minute cooldown in `Reminders Memory` (approximately 6.25 days) to prevent double-sends.

`runWeekendPlanner_()` assembles clear weekend windows (via `findClearWindows_()` filtered to windows that touch a Friday or Monday), computes the intensity signal, reads goals, the Shared Interest Ledger, and PTO stats, then calls Claude to generate a "Weekend Decision Memo" with three archetypes:

- **THE EXTENSION**: goal-anchored activity
- **THE CONTRAST**: rest/recharge (weighted heavier if intensity is high)
- **THE PROTOTYPE**: new experience not in the Interest Ledger

The memo is delivered via `sendNudge_()` and also written as an all-day Google Calendar event on the upcoming Saturday in the `pto_vera_calendar` (default: `'Vera'` calendar).

---

8. Chat System Architecture

Context assembly

Every chat turn in `Chat.js` assembles a fresh context object via `buildChatContext_()`. The context includes:

- Active flags (unresolved, unacknowledged) from the Flags tab
- Open tasks from `getOpenTasks()`
- Upcoming events from `getUpcomingEvents()`
- Life summaries from `getSummaries()`
- Bills from the Bills tab
- Home items from the Home Items tab
- Goals from `getGoals_()`
- Financial goals
- Projects
- Travel context (upcoming trips, packing items by trip key, itinerary items by trip key)
- Career position, goals, progression, development, wins, network
- And more domain-specific data depending on the chat turn context

This context is assembled fresh on every turn — there is no persistent in-memory state between chat calls (Apps Script does not support persistent processes).

`buildChatSystemPrompt_(context)`

The system prompt injects the assembled context as structured sections for Claude to reason over. It also injects:

- Proactive insights from `computeProactiveInsights_(context)` — surfaced as "VERA NOTICES" in the prompt
- Source routing identity (Ahmed vs Victoria vs Slack)
- Persona definition and action capabilities

Proactive insights — `computeProactiveInsights_(context)`

Pre-computes from context data (no additional API calls) and returns a formatted `[HIGH/MEDIUM/LOW]` notice string. Sources checked:

1. Tasks overdue > 3 days (by name, sorted by staleness, top 3)
2. Bills due within 7 days and unpaid (urgency: `<= 2 days = HIGH`, `<= 5 = MEDIUM`, `<= 7 = LOW`)
3. Upcoming trips within 14 days: empty packing list (`HIGH` if within 7 days), unpacked items (`MEDIUM`), empty itinerary (`MEDIUM`)
4. Home items with service overdue (`HIGH`) or due within 7 days (`LOW`)
5. Projects with overdue tasks (`MEDIUM`)

These are injected into the system prompt so Claude can surface them proactively at the start of a conversation or when contextually relevant.

Source routing

The `body.source` field determines how Claude identifies the conversation participant:

- `'dashboard'`: Ahmed (primary user)
- `'dashboard-lite'`: Victoria (simplified dashboard)
- `'vera-chat'`: Slack `#vera-chat` channel — identity derived from Slack user ID via `getSlackUserName_()`

Action dispatch

Chat.js contains a large `dispatchAction_()` function that handles Claude's structured action responses. Claude returns a JSON action object (`{ action, params }`) alongside its conversational reply. Supported actions include (but are not limited to): `add_task`, `complete_task`, `resolve_flag`, `snooze_flag`, `acknowledge_flag`, `update_goal`, `add_trip_item`, `add_packing_item`, `log_gym_session`, `add_bill`, `add_home_item`, `update_project_task`, `record_purchase`, and many more. Each action modifies the appropriate sheet tab directly.

Multi-turn context

Chat history is stored per session in Script Properties under the key `CHAT_HISTORY_{sessionId}` as a JSON-serialised array of `{ role, content }` messages. The `CHAT_MAX_EXCHANGES` constant (default: 10) limits the stored back-and-forth to 20 messages (10 pairs). Older messages are trimmed from the front of the array.

Web search

When `VERA_SEARCH_API_KEY` is configured in Script Properties, the `WEB_SEARCH_TOOL_` is appended to the Claude API call's `tools` array. Claude can invoke it for real-time or location-specific queries. A PII scrub (email, phone, SSN regex) runs on every query before it leaves Apps Script. The engine is configurable via `VERA_SEARCH_ENGINE` Script Property (`'serper'` or `'tavily'`, default `'serper'`).

---

9. Slack Integration Architecture

Inbound (user → VERA)

Slack sends `POST` requests to the Apps Script web app `doPost()` endpoint via the Events API. Two payload shapes arrive:

1. **JSON body** (`body.type === 'event_callback'`): routed to `handleSlackEvent_()` in `Slack.js`
2. **Form-encoded body** (button interactions, slash commands): routed to `handleSlackFormPost_()` in `Slack.js`

Apps Script does not reliably decode form-encoded POST bodies into `e.parameter`. `handleSlackFormPost_()` falls back to manually URL-decoding `e.postData.contents` when `e.parameter` is empty.

Bot message filtering: `handleSlackEvent_()` checks `event.bot_id` and `event.subtype === 'bot_message'` before processing. VERA's own outbound messages arrive back via the Events API and must be ignored to prevent echo loops.

Slack user allowlist: `getSlackAllowedUserIds_()` reads `SLACK_ALLOWED_USER_IDS` from Script Properties (comma-separated Slack user IDs). Only allowlisted users can trigger VERA via Slack. Unknown users receive a silent rejection.

Async queuing: Chat messages from `#vera-chat` are queued in `CacheService` and processed asynchronously. This is required to respond within Slack's 3-second acknowledgement deadline — the Apps Script execution (which involves calling Claude) can take 10–30 seconds.

Outbound (VERA → user)

Three send functions handle outbound messages:

- `sendSlackMessage_(channelId, text, blocks, threadTs)`: Generic send to any channel. Accepts optional Block Kit `blocks` array and optional `threadTs` for threading.
- `sendSlackNotification_(text, blocks, urgency)`: Routes to `#vera-notifications` (`SLACK_NOTIFICATIONS_CHANNEL_ID`). For `urgency === 'High'`, prepends `<@AHMED_USER_ID>` to trigger a Slack mention.
- `sendSlackLog_(text)`: Plain text to `#vera-logs` (`SLACK_LOGS_CHANNEL_ID`). Called by `nightlyRun()`, reminder delivery, pattern recognition, travel pipeline, and monthly review.

Block Kit

`buildFlagBlocks_(flag)` assembles a three-block structure for each flag:

1. `section` block with urgency emoji + bold flag title + italic reason (truncated to 200 chars)
2. `actions` block with two buttons: `acknowledge_flag` (action_id) and `snooze_flag` (action_id), each carrying the flag ID as the `value`
3. `divider` block

When a user clicks Acknowledge or Snooze, Slack sends an interaction payload (form-encoded JSON) to `doPost()`. `handleSlackInteraction_()` parses the action ID and value, updates the flag in the sheet, and calls `sendSlackResponse_(responseUrl, confirmText, null, true)` with `replace_original: true` to replace the button message with a confirmation.

The morning nudge uses `buildMorningNudgeBlocks_()` which groups flags by urgency (High → Medium → Low) and includes a capacity mode ticker and a "held from yesterday" count.

Channel routing

Channel	Script Property key	Purpose
`#vera-chat`	`SLACK_CHAT_CHANNEL_ID`	Bidirectional conversation (Ahmed + Victoria)
`#vera-notifications`	`SLACK_NOTIFICATIONS_CHANNEL_ID`	Flag alerts with Acknowledge/Snooze buttons
`#vera-logs`	`SLACK_LOGS_CHANNEL_ID`	System event log — nightly run, feature fires, errors

---

10. Travel Pipeline

Trip detection

Trips are identified by the `Itinerary` tab. Each row has a `Trip Key` column with the format `YYYY-MM-DD|Trip Label` (e.g. `2026-06-15|Nashville Weekend`). The departure date is the `YYYY-MM-DD` prefix. The `TripMeta` tab stores per-trip context: a traveler field identifies which trips are Ahmed-only, Victoria-only, or joint. Victoria-only trips are excluded from `checkVacationMode_()` and the pacing system.

Email parser — `runEmailScan_()`

Runs every 30 minutes via the `runEmailScan_` trigger (only when `email_parser_enabled = 'true'` in Config). The process:

1. Runs a broad Gmail search via `buildTravelSearchQuery_()` — returns up to 50 threads
2. Deduplicates against the `Processed Emails` tab (`isAlreadyProcessed_(messageId)`)
3. Batches new candidates (up to `BATCH_SIZE = 20`) to Claude with a confidence scoring prompt
4. Per-email response: `confidence >= HIGH_CONF (0.85)` → auto-process (write/enrich itinerary row); `LOW_CONF (0.60) <= confidence < HIGH_CONF` → hold and write a flag for manual confirmation; `confidence < LOW_CONF` → discard silently
5. Marks every processed email in `Processed Emails` tab with outcome and mode

The email body is truncated to `MAX_SIGNAL_CHARS = 2500` characters before sending to Claude. The warning in `setupTriggers()` notes that with emails per day up to the 50-thread search limit and 30-minute intervals, this could generate up to 144 Claude API calls per day — the config key `email_parser_enabled` defaults to `'false'`.

Pre-trip briefing — `checkPreTripBriefings_()`

Reads the Itinerary tab, finds trips departing within `pretrip_briefing_hours` (default: 48 hours), and assembles a `buildPreTripBriefingFlag_()` for each. The briefing flag is written as `urgency: 'High'` with source `'Trip Briefing'`. Deduplication via `writeFlags()` key fingerprinting ensures one briefing per trip, ever — the key includes

the trip key, so it fires once and never again even if the nightly run repeats. Logged to `#vera-logs`: `:airplane: Pre-trip briefing written — Nashville Weekend`.

Flight status monitor — `checkFlightStatuses_()`

Polls the AviationStack API every 15 minutes via the `checkFlightStatuses_` trigger. Active only for flights within 24 hours of departure. Polling intervals are rate-limited:

- Departure > 6h away (in the 6–24h window): every 3 hours
- Departure 1–6h away: every 60 minutes
- Departure < 1h away: every 15 minutes

Status is stored in: (a) the Itinerary tab row's `Metadata` column (JSON with `flight_status` key) for sheet-sourced flights; (b) Script Properties under `FLIGHT_STATUS_CACHE` (keyed by `CAL-xxx` IDs) for calendar-sourced flights.

When a 429 response is received from AviationStack: a monthly quota error backs off for 30 days; a rate-limit error backs off for 2 hours. The backoff window is stored in `AVIATIONSTACK_BACKOFF_UNTIL` Script Property and checked at the top of every poll.

Post-trip capture — `checkPostTripCapture_()`

Fires `posttrip_capture_delay_days` (default: 1) after a trip's end date. Writes a flag (source: `'Trip'`) prompting Ahmed to log memories, expenses, and notes. Gated by `posttrip_capture_enabled = 'true'`.

---

11. PTO & Clear Windows

PTO snapshot — `writePTOSnapshot_()`

Reads Ahmed's work calendar (configured via `pto_calendar_name` Config key, default `'Verizon Calendar'`). Classifies all-day events:

- Title contains `'Vacation'` → vacation day (pool: `pto_vacation_days`, default 20)
- Title contains `'PTO'` (all-day) → 8 personal hours (pool: `pto_personal_hours`, default 48)
- Title contains `'PTO'` (timed) → actual event duration in hours
- Title matches `pto_holiday_keywords` (default: `'Day,Holiday,Floating,Closure'`) → company holiday (not counted against PTO pools)
- Title matches `pto_ignore_keywords` (default: `'Pay Day'`) → skip entirely

Computes: days used, days remaining, ideal burn-rate pace (remaining days / remaining weeks in year), pace gap (actual vs ideal). Writes `[AUTO]` rows to the Summaries tab covering vacation balance, personal time balance, and pace status. Returns `ptoStats` object passed to Monthly Review.

Clear windows — `findClearWindows_()`

`findClearWindows_(gapCalendars, today, lookAheadDays, minWorkdays)` scans the next `lookAheadDays` days and finds contiguous stretches of `minWorkdays` or more clear workdays. "Clear" means no events on calendars defined by `pto_gap_calendars` Config key (default: `'Verizon Calendar,AE&VV - Our Joint Chaos'`).

Used by `getWeekendWindows_()` in `WeekendPlanner.js` (filtered to windows touching a Friday or Monday) and by the Health Tracker to suggest appointment booking windows.

3-2-1 Framework

The PTO system surfaces a 3-2-1 planning framework in the dashboard PTO tab:

- **3 months out**: Plan big trips and time-off blocks
- **2 weeks out**: Confirm logistics, book details
- **1 day out**: Verify everything is ready

The `ptoSummaryForClaude_()` function formats PTO data for injection into the Claude prompt, including 3-2-1 status indicators and a `paceStatus` field (`behind`, `on_track`, `ahead`).

---

12. Summaries Architecture

Two-tab architecture

Metrics tab (`TABS.METRICS`): VERA's self-monitoring. Written by `writeMetrics_(ss)`. Contains task counts (open, overdue, neglected), calendar event counts (today, this week), and flag counts (total, by urgency). Fully automatic — no user configuration needed. Rows are `[AUTO]`-prefixed and rewritten every night.

Summaries tab (`TABS.SUMMARIES`): External life intelligence. Written by `writeSummaries_(ss)`. Populated from external Google Sheets via `summary_sheet:SourceName` Config rows. Also receives `[AUTO]` rows from PTO snapshot, fitness data, and other module writes.

AUTO rows

Any row whose `Source` column starts with `[AUTO]` is owned by the script. The `clearAutoRows_(sheet, tabName)` function:

1. Reads all rows in the sheet
2. Collects row indices where `Source` starts with `'[AUTO]'`
3. Deletes them from bottom to top (to preserve row indices during deletion)

After clearing, new `[AUTO]` rows are inserted before row 2 (just after the header) and styled with `background: '#eef2ff'` (subtle blue tint) and `fontColor: '#333333'`. Manual rows (without `[AUTO]` prefix) are never touched.

External sheet hooks

Config tab syntax:

```
Setting                          | Value
summary_sheet:SimpleAssTracker   | SHEET_ID|TabName|CellRef|metric_name
```

For example:

```
summary_sheet:SimpleAssTracker   | 1aBc...XyZ|Budget|B2|checking_balance
summary_sheet:SimpleAssTracker   | 1aBc...XyZ|Budget|B5|groceries_actual_vs_budget
summary_sheet:Fitness            | 1xYz...AbC|Log|D4|gym_sessions_this_week
```

`writeSummaries_()` reads all Config rows with keys starting with `summary_sheet:`. For each, it opens the external sheet via `SpreadsheetApp.openById(sheetId)`, reads the specified cell, and writes an `[AUTO]` row to the Summaries tab: `source = '[AUTO] SourceName'`, `metric = metric_name`, `value = cellValue`, `asOf = today`.

The external sheet must be shared (at minimum viewer access) with the Google account running the script.

`getSummaries()`

Reads both the Metrics tab and the Summaries tab and returns a merged array of `{ source, metric, value, asOf }` objects. This merged array is what `generateFlags()` and chat context assembly receive — they do not distinguish between the two tabs.

---

13. Monthly Life Review

Trigger

`checkMonthlyReview_(ptoStats)` is called from `nightlyRun()` Step 0o. The function immediately returns if `today.getDate() !== 1` — it only runs on the 1st of each month. A second guard checks whether a review with the key `monthly_review_YYYYMM` (for the prior month) already exists anywhere in the Flags tab (any status) — if found, it returns without re-generating. This dedup ensures the review fires exactly once per month even if `nightlyRun()` somehow triggers multiple times on the 1st.

The Config key `monthly_review_enabled` (default `'true'`) gates the entire feature.

Assembly — `buildMonthlyReview_(label, priorMonth, ptoStats)`

Assembles the review by calling independent section builders, each wrapped in `try/catch`:

- **Goals**: Current-year goals grouped by status (Done, Doing, Stalled, Backlog)
- **Tasks**: Open task count, overdue count, most neglected tasks by age
- **Finance**: Reads spending data from the Summaries tab — `[AUTO]` rows with finance/SAT source
- **PTO**: Uses the `ptoStats` object passed from Step 0b (avoids re-reading the calendar)
- **Travel**: Reads the Itinerary tab for trips whose date range overlaps the prior month
- **Flags**: Counts flags generated in the prior month, resolved vs still-open

- **Carry Forward**: A single Claude call asking for one insight to carry forward from the month's data

Output

Three outputs are produced:

1. **Flag**: Written via `writeFlags()` with `source: 'Monthly Review'`, `urgency: 'Low'`, `key: 'monthly_review_YYYYMM'`. The full review text goes in the `reason` field.
2. **Archive**: Appended to the `Monthly Reviews` tab (headers: `Month Key | Month Label | Generated Date | Review`) as an append-only record. This tab is never modified by VERA — only appended.
3. **Email**: Sent via `sendMonthlyReviewEmail_(label, reviewText)` as a styled HTML email.
4. **Slack log**: `sendSlackLog_('■ Monthly review written — March 2026')`.

---

14. Configuration Architecture

Config tab (runtime-tunable)

The `Config` tab in the Life OS sheet contains key-value pairs read by `getConfigValues()` on every execution:

```
function getConfigValues() {
  if (_configCache_) return _configCache_;  // per-execution cache
  var ss    = getSpreadsheet();
  var sheet = ss.getSheetByName(TABS.CONFIG);
  // ... reads all rows into an object ...
  _configCache_ = cfg;
  return cfg;
}
```

`_configCache_` is a module-level variable declared as `var _configCache_ = null` in `Code.js`. It resets automatically at the start of each Apps Script execution (there is no persistent memory between executions). Within a single execution (e.g. a nightly run), `getConfigValues()` is called 4–6 times — the cache prevents the Config tab from being re-read on every sub-function call.

Config keys that change behaviour have sensible defaults hard-coded as fallback strings (e.g. `cfg['calendar_days_ahead'] || '7'`). This means the system works out of the box with an empty Config tab; users only add rows to override defaults.

Config tab keys used across the system include:

Key	Default	Module	Purpose
`calendar_days_ahead`	`'7'`	Calendar.js	Look-ahead window for events
`task_age_threshold_days`	`'7'`	Tasks.js	Age before a task is "neglected"

Key	Default	Module	Purpose
`max_flags_per_night`	`'8'`	Claude.js (`CONFIG.MAX_FLAGS`)	Cap on Claude flag output
`skip_calendars`	`'Holidays in United States'`	Calendar.js	Calendars to exclude entirely
`calendar_label:CalName`	(none)	Calendar.js	Custom label for a calendar name
`pto_vacation_days`	`'20'`	PTO.js	Annual vacation allocation
`gym_sessions_per_week`	`'3'`	PatternRecognition.js, GymTracker.js	Weekly gym target
`reminders_enabled`	`'true'`	Reminders.js	Master switch for Anticipator
`explorer_enabled`	`'true'`	Reminders.js	Master switch for Explorer
`email_parser_enabled`	`'false'`	EmailParser.js	Master switch for inbox scanner
`pretrip_briefing_enabled`	`'true'`	PreTripBriefing.js	Enable/disable pre-trip flags
`monthly_review_enabled`	`'true'`	MonthlyReview.js	Enable/disable monthly review
`pattern_max_flags`	`'2'`	PatternRecognition.js	Max cross-domain flags per run
`pattern_dedup_days`	`'7'`	PatternRecognition.js	Dedup window for pattern flags
`pacing_enabled`	`'true'`	Pacing.js	Enable/disable pacing system
`pacing_flag_threshold`	`'3'`	Pacing.js	Unacked flags to trigger miss scoring
`summary_sheet:SourceName`	(none)	Summaries.js	External sheet hook definition

Script Properties (deployment secrets)

Read by `PropertiesService.getScriptProperties()`. These are set once in the Apps Script editor (Project Settings → Script Properties) and never committed to source control.

Property	Used by	Purpose
`VERA_SHEET_ID`	`Code.js` (`CONFIG.SHEET_ID`)	The Life OS Google Sheet ID
`MORNING_NUDGE_EMAIL`	`Code.js`	Email address for error notifications and email-mode nudges

Property	Used by	Purpose
`CLAUDE_API_KEY`	`Claude.js`	Anthropic API key
`SLACK_BOT_TOKEN`	`Slack.js`	Slack bot OAuth token
`SLACK_CHAT_CHANNEL_ID`	`Slack.js`	`#vera-chat` channel ID
`SLACK_NOTIFICATIONS_CHANNEL_ID`	`Slack.js`	`#vera-notifications` channel ID
`SLACK_LOGS_CHANNEL_ID`	`Slack.js`	`#vera-logs` channel ID
`SLACK_AHMED_USER_ID`	`Slack.js`	Ahmed's Slack user ID (for @mentions)
`SLACK_VICTORIA_USER_ID`	`Slack.js`	Victoria's Slack user ID
`SLACK_ALLOWED_USER_IDS`	`Slack.js`	Comma-separated allowlist of Slack user IDs
`AVIATIONSTACK_KEY`	`FlightStatus.js`	AviationStack API access key
`VERA_SEARCH_API_KEY`	`Chat.js`	Serper.dev or Tavily API key for web search
`VERA_SEARCH_ENGINE`	`Chat.js`	`'serper'` or `'tavily'`
`PACING_MODE_ACTIVE`	`Pacing.js`	Runtime state flag
`PACING_MODE_ENDS`	`Pacing.js`	Runtime state — expiry date
`VACATION_MODE_ACTIVE`	`Pacing.js`	Runtime state flag
`VACATION_MODE_ENDS`	`Pacing.js`	Runtime state — trip end date
`VACATION_TRIP_NAME`	`Pacing.js`	Runtime state — active trip key
`VERA_LOGO_FILE_ID`	`Code.js` (morningNudge)	Google Drive file ID for email logo

Decision rule for where a value lives

• **Secret or credential** (API key, channel ID, user ID, sensitive token) → Script Property. Never in the sheet, never in source control.

• **Value the user might want to tune** (thresholds, labels, calendar names, feature flags) → Config tab with a sensible default fallback in code.

• **Value that changes system behaviour** (feature on/off switches, interval days, target counts) → Config tab with a sensible default fallback. The fallback ensures the system works without any Config row being present for that key.

• **Value that is computed at runtime and needs to survive between executions** (pacing mode active/inactive, vacation mode, AviationStack backoff window) → Script Properties as ephemeral state. These are not secrets but they need to persist across the 6-minute execution boundary.

The Config tab's layout (November 2026)

`getConfigValues()` reads columns A and B only, keyed by the name in column A. Rows whose column A is blank are skipped, and a section title row is just a key with an empty value. So the tab can be laid out for people without the code noticing:

- **Columns:** A *Setting* · B *Value* (the only column VERA reads) · C *What it does* · D *Where it's used* (a `=HYPERLINK("#gid=…")` to the related tab).
- **Order:** a Contents block of links (rows with an empty column A), then system-wide sections (General, Notifications, Calendars, Linked files, Assistants), then one section per feature. Each section is a title row plus a collapsible row group.
- **Defaults made visible:** settings the code reads with an inline fallback (`cfg['x'] || 'default'`) are listed with that same value and "(default)" in column C, so changing them is a matter of editing B. A row whose value equals the code default behaves exactly like a missing row.
- **Writers** (`Groceries.js`, `PTO.js`, the notification channel route in `WebApp.js`, `ComingUp.js`) all find their row by key and write column B, or append; none depend on row positions.
- **Data tabs are different:** they are read by position (`getRange(2, 1, n, HEADERS.length)`), so columns there are never moved or inserted. Formatting (colours, banding, frozen rows, number formats on numeric cells, tab order and tab colours) is safe; issue #235 turns the sheet styling into a re-runnable script.

---

15. Trip Identity, Emails & Readiness

Trip registry (`Trips.js`). Every trip gets an immutable `TRIP-…` ID in the *Trips* tab. Label and dates track the calendar; earlier keys (`startDate|label`) are kept in *Aliases*. Resolution is layered: calendar event ID, then alias, then label + date proximity (14 days), never a guess when two candidates tie. Every read of a trip-keyed row goes through `tripKeyMatches_` / `tripKeysFor_`, so rows written under an old key still belong to the trip after a date change. Daily latches (Travel Day email, pre-trip briefing) key on the Trip ID, so a moved trip never emails twice.

Who's travelling and Travel Day recipients (`TripEmails.js`). TripMeta's *Traveler* column holds names from the traveller profiles (old one-word values such as `Both` still read). The Trips tab's *Email Recipients* holds the switched-on travellers' names and any added addresses, separated by `;`. `teResolve_` builds the To list: the owner always on trips he travels on; Victoria only when switched on, with her address from Config `victoria_email`; others from their profile email; added addresses as given. Victoria's solo trips send no Travel Day email. Config `travel_companions` is retired.

Traveller profiles (`TravelerProfiles.js`). The *Traveler Items* tab holds one row per document, membership, preference or health item per person. The server keeps only the last 4 digits of any number. Held visas and ETAs mark a destination as covered in the documents check; a nightly pass reminds about passports, ETAs and trusted-traveller expiries ahead of trips (notification `travel_docs`).

Readiness (`TripReadiness.js`). `trdReadiness_` turns a trip, its itinerary metadata, documents, packing and prescriptions into rows per traveller: ready / to do / blocks (documents, member number on bookings, seat preference, lounge access by status or card, refills due before the trip ends).

16. Packing, Luggage & Leaving Home

• `Luggage.js` **(#230):** "Our bags" (size, carry-on/checked, empty weight), per-person bag advice from nights, weather, trip type and the list, about-weights against the fare's allowance, and which status or card waives the checked bag. Road trips get bag advice only.

• `LeavingHome.js` **(#231):** checklist templates per home, filtered by season, length and who stays; runs per trip or for ad-hoc "Heading out" time away. Ticks are shared and record who ticked. The night-before email carries the list; a morning check sends `leave_home` if items remain.

• `PackPlus.js` **(#234):** status per item (`To buy → Ready → In the bag`), quantities, who brings it, staples (`always_pack` profile items, offered after generating), copy from a similar past trip, outfits per day, a Shopping "Trip: …" buy list, a checkout tick-off, and a post-trip review that feeds the next list.

17. Finances (November 2026)

• **Budget (`Budget.js`):** reads the budget tracker's Tracker tab live and by label; works out plan vs actual and a What-if overlay without writing the tracker.

• **Bills & cashflow (`BillSchedule.js`):** the Bill Schedule tab (due day, account override, autopay, card) joined to the tracker's bills, and a joint-account timeline with the opening balance and transfer day.

• **Summary & check-in (`FinSummary.js`):** quarterly snapshots (*Finance Snapshots*), decisions with owners (*Check-in Decisions*), notes, and the `quarterly_checkin` reminder.

• **Groceries and receipt runs (`Groceries.js`, `GroceryRun.js`):** a receipt photo is read by Claude vision (store, date, lines, total); picked lines go to Purchase History (On Hand syncs from it), matching shopping-list items are ticked across stores, and the run is logged to *Grocery Runs* with its date.

• **Phone plan, Planned Purchases, planner Goals and Net worth** read their own sheets; the planner is read live and never written.

18. Places, Gifts, Life & Career

• **Places (`Places.js`, `BucketPlus.js`):** Countries (map/list/timeline, occasions) and the Bucket List with things to do, best months (cached per place in *Place Facts*), vacation-day cost, award points from Config `award_*` rows, and a long-weekend filter by direct flight time from `home_airports`.

• **Gifts (`TripGifts.js`):** kind, status, budget, history per person, ideas per destination (cached in *Gift Place Ideas*) and customs notes.

• **Life (`Life.js`, `LifeGrowth.js`, `Growth.js`, `ComingUp.js`, `QuickCapture.js`):** Explore, Growth (Now, Goals, Habits, Learning, Trials, Review), Coming up (calendar events by type with lead days from `coming_up_type:*`), and Quick capture, which classifies one line of text and files it with an Undo.

• **Career Credentials (`CareerCredentials.js`):** certifications and degrees with expiry and renewal credits; reminders at 90/30/7 days and behind-pace (`career_cert_expiry`); shared with the job agent.

19. Family Planning & the Cycle Tracker

`FamilyPlanning.js` holds one shared plan (items, leave blocks, policy) by phase. `Cycle.js` adds a *Cycle Log* (start, end, who logged). Pure functions compute stats (average of the last six gaps, outliers ignored), today's view (cycle day, mood note from `cycle_mood_days`, next period, fertile window and peak days) and due notices. Notices are sent only

while the phase is Trying for the fertile window, once per key, through `cycle_reminders` to a channel chosen in System › Notifications. Nothing goes on the shared flag list.

20. Dashboards, Navigation & Notifications

- **Two dashboards:** `docs/index.html` (built from `docs/app.js` by `docs/build.js`, React inlined) and `docs/dashboard-lite.html` (Victoria's, compiled in the browser). Shared blocks are copied byte-for-byte between them and tests enforce it.
- **Tile navigation:** each area groups its sub-tabs into tiles (`VN_GROUPS`, `FIN_GROUPS`, `HF_GROUPS`, `LF_GROUPS`, `SYS_SUBS`). The lite dashboard has eight tabs: Today, Chat, Shopping, Home, Travel, Money, Family, Projects.
- **Crash screen (planned release):** a `VeraBoundary` error boundary around the app and each tab, and a head boot guard that explains an empty page, replace the old blank white screen; lite pins its React and Babel versions.
- **Notifications:** every message is listed in `NOTIF_REGISTRY` (`Code.js`) with a default channel; Config `notif_<key>_channel` and `notif_<key>_enabled` override it, and System › Notifications edits them.

---

21. Appendix: Developer Reference

> *Carried over from the old developer cheat sheet (last updated November 1, 2026). Modules added since are covered in sections 15–20.*

Triggers setupTriggers() in Code.js

Function	Schedule	Purpose
`nightlyRun`	Daily 11 PM	Main intelligence pipeline — summaries, PTO, patterns, Claude flags, Slack log
`morningNudge`	Daily 7 AM	Morning email digest with active flags, tasks, calendar preview
`hourlyCheck`	Every 1 hour	Anticipator reminder rules (ergonomic, hydration, bills, packing) + Weekend Planner on Monday
`checkFlightStatuses_`	Every 15 min	Flight status monitoring via AviationStack — only active when trips are within 24h of departure
`runEmailScan_`	Every 30 min	Travel email parser — gated: `email_parser_enabled=true`. ■ Up to 144 Claude API calls/day when enabled

Nightly Pipeline Steps nightlyRun() in Code.js

Step	Function	Description
-1	`escalateAgedFlags_`	Escalate unacknowledged flags aged 3d → "3d", 7d → "7d" in Escalated column
0	`writeSummarySnapshot`	Auto-populate Metrics + Summaries tabs from live data (Phase 5)
0a	`syncCalendarBirthdaysToImportantDates_`	Sync upcoming birthdays from Joint Chaos calendar → Important Dates tab
0a-ii	`resetChoresByCadence_`	Reset household chores checkboxes based on cadence (daily/weekly/monthly/etc.)
0b	`writePTOSnapshot_`	Write PTO tab snapshot + compute VERA calendar recommendations
0c	`runExplorer_`	Daily AI discovery bulletin — sends to Slack/email based on `explorer_interests`
0d	`getSuppressedKeyPatterns_`	Signal Learning — load noise-suppressed patterns to filter from Claude's output
0e	`recordExpiredFlags_`	Signal Learning — record flags open 30+ days without action as "expired"
0f	`checkPreTripBriefings_`	48-hour pre-trip auto-briefing flags (weather, packing status, flight info)
0g	`checkPostTripCapture_`	Post-trip debrief prompt — fires N days after trip end
0h	(inline)	Reset Morning Routine checkboxes (Checked=false, Checked At=blank)
0i	`checkGymSessions_`	Gym session check-in — scans for ended EXERCISE-tagged calendar events
0j	`checkFitnessConsistency_` / `checkFitnessTravelGap_`	Weekly gym consistency check + travel gap gap detection
0k	`autoRestockItems_` / `generatePantryFlags_`	Pantry: auto-restock prediction + trip-overlap flags (gated: `pantry_enabled=true`)
0l	`inferCapacityMode_`	Score tomorrow's calendar load → infer busy/normal/light mode
0m	`checkContracts_`	Contract expiry flags — upcoming renewals/expirations in Contracts tab

Step	Function	Description
0n	`checkHealthAppointments_`	Health appointment due-date checks — reads DR:-prefixed calendar events
0o	`checkMonthlyReview_`	Monthly Life Review — fires on 1st of month (gated: `monthly_review_enabled=true`)
0p	`resetWeekMealPlan_`	Saturday only: archive current meal week, seed next week
0q	`checkCrossPatternFlags_`	Cross-domain pattern recognition — compound signals across all domains (PatternRecognition.js)
1	`getUpcomingEvents / getOpenTasks / getSummaries / getSharedInterestLedger_`	Collect live data for Claude prompt
1b	`suggestDueDates`	AI-suggest due dates for undated tasks — writes back to Tasks tab
2–3	`generateFlags`	Build Claude prompt → call claude-sonnet-4-6 → parse flags JSON
4	`writeFlags`	Append flag rows to Flags tab with dedup (exact + fuzzy key matching)
4b	`recordFlagsGenerated_`	Signal Learning — record generated flag keys as "seen"
end	`sendSlackLog_`	Post nightly summary to #vera-logs (flag counts, step warnings, elapsed time)

Flag Sources source: values written by writeFlags()

Source Value	Module / Step	What it generates
`Calendar` / `Tasks` / `Summaries`	Claude.js	Primary nightly Claude output — cross-domain analysis
`Health Tracker`	HealthTracker.js	Overdue / upcoming health appointments (DR: prefix events)
`Fitness Tracker`	Fitness.js	Weekly gym consistency and travel gap flags
`Gym Tracker`	GymTracker.js	Post-session check-in prompts
`Pattern Recognition`	PatternRecognition.js	Cross-domain compound pattern flags (all 7 patterns)
`Pre-Trip Briefing`	PreTripBriefing.js	48h pre-departure briefing

Source Value	Module / Step	What it generates
`Post-Trip Capture`	PostTripCapture.js	Post-return memory capture prompt
`Email Parser`	EmailParser.js	Parsed travel confirmations from inbox
`Monthly Review`	MonthlyReview.js	Monthly life review flag (1st of month)
`Important Dates`	ImportantDates.js	Upcoming birthdays, anniversaries (30-day lead)
`Contracts`	Code.js (checkContracts_)	Contract renewal / expiry alerts
`Pantry Tracker`	Pantry.js	Purchase-history restock flags (gated)
`Finance`	FinancialGoals.js	Financial goal milestone / savings rate alerts

Urgency	Visual	Sheet Background	Meaning
High	■	`#fee2e2` (red tint)	Needs attention soon — @mention in Slack
Medium	■	`#fef9c3` (yellow tint)	Review this week
Low	■	`#dcfce7` (green tint)	Good to know, no urgency

Calendar Event Prefixes Special title prefixes parsed by VERA

Prefix	Example Title	Detected By	Effect
`DR:`	`DR: Ahmed - Annual Physical` DR: [Person] - [Type] - [Dr. Name (opt)]	HealthTracker.js	Tracks last-visit date per person+type, computes next-due, generates flag when overdue/approaching
`EXERCISE` (in description)	Any title with `EXERCISE` in the event description	GymTracker.js	Logs attended gym session to Gym Log tab; generates check-in flag
`TRIP:` (convention)	`TRIP: Paris - CDG → ORD`	FlightStatus.js, PreTripBriefing.js	Trip tracking — pre-trip briefing, packing, itinerary, post-trip capture. Any multi-day event works.
`calendar_label:*`	Config row, not an event prefix	Calendar.js	Maps calendar name to a display label for Claude prompt context

Config Tab Keys All cfg['...'] lookups — Setting column in Config sheet

Key	Default	Module	What it controls			
calendar_days_ahead	7	Calendar.js	Days of calendar events to fetch for nightly analysis			
task_age_threshold_days	7	Tasks.js / Code.js	Days before an open task is considered "neglected"			
max_flags_per_night	8	Claude.js	Maximum Claude-generated flags per nightly run			
morning_nudge_time	7	Code.js	Hour (24h) for morning nudge trigger			
snooze_default_days	2	WebApp.js	Default snooze duration in days			
finance_review_day	1	Code.js	Day of month for finance review			
active_sources	Calendar,Tasks,Summaries	Code.js	Comma-separated data sources included in Claude prompt			
skip_calendars	Holidays in United States	Calendar.js, HealthTracker.js, GymTracker.js	Comma-separated calendar names to ignore entirely			
victoria_calendars	blank	Calendar.js, WebApp.js	Comma-separated calendar names belonging to Victoria			
calendar_label:*	none	Calendar.js	Maps calendar name to display label (e.g. `calendar_label:Ahmed = personal`)			

Key	Default	Module	What it controls			
pto_vacation_days	20	PTO.js	Annual vacation allocation in days			
pto_rollover_days	0	PTO.js	Days carried over from prior year			
pto_personal_hours	48	PTO.js	Annual personal time in hours			
pto_buffer_days	3	PTO.js	Reserve days held back from planning calculations			
email_parser_enabled	false	EmailParser.js	Enable 30-min inbox scan for travel confirmations. ■ Up to 144 Claude calls/day			
email_admin_enabled	false	EmailAdmin.js	Enable weekly inbox triage + follow-up tracking			
email_admin_frequency	weekly	EmailAdmin.js	`weekly` or `3days`			
pretrip_briefing_enabled	true	PreTripBriefing.js	Generate 48h pre-departure briefing flags			
pretrip_briefing_hours	48	PreTripBriefing.js	Hours before departure to generate briefing			
posttrip_capture_enabled	true	PostTripCapture.js	Enable post-trip memory capture prompt			
posttrip_capture_delay_days	1	PostTripCapture.js	Days after trip end to fire capture flag			

Key	Default	Module	What it controls			
gym_tracker_enabled	true	GymTracker.js	Enable gym session tracking from calendar EXERCISE tags			
gym_tracker_lookback_hours	24	GymTracker.js	Hours to look back for ended EXERCISE calendar events			
gym_sessions_per_week	3	PatternRecognition.js	Target gym sessions per week for pattern detection			
fitness_enabled	false	Fitness.js	Enable weekly gym consistency check flags			
fitness_weekly_target	4	Fitness.js	Target gym sessions per week for consistency flags			
fitness_low_flag_day	4	Fitness.js	Day of week to fire Low flag if behind (1=Sun … 7=Sat; 4=Wed)			
fitness_travel_block_time	07:00	Fitness.js	Start time for auto-created trip gym session blocks			
fitness_travel_block_duration	60	Fitness.js	Duration (minutes) for auto-created trip gym session blocks			
pacing_enabled	true	Pacing.js	Enable automatic pacing mode detection and suggestions			

Key	Default	Module	What it controls			
pacing_flag_threshold	3	Pacing.js	Minimum High flags to trigger pacing mode offer			
pacing_mode_days	7	Pacing.js	Duration (days) for pacing mode when activated			
reminders_enabled	true	Reminders.js	Master switch for Anticipator reminder rules			
explorer_enabled	true	Reminders.js	Master switch for Explorer daily AI discovery			
explorer_interests	(built-in default)	Reminders.js	Interest topics injected into Explorer AI prompt			
ergonomic_interval_min	60	Reminders.js	Ergonomic break target interval in minutes			
hydration_interval_min	120	Reminders.js	Hydration reminder interval in minutes			
mobility_reminder_hour	20	Reminders.js	24h hour for evening mobility nudge			
weekend_planner_enabled	true	WeekendPlanner.js	Enable Monday morning Weekend Planner Slack message			
weekend_planner_hour	8	Reminders.js, WeekendPlanner.js	Hour to fire Weekend Planner (fires from hourlyCheck on Monday)			

Key	Default	Module	What it controls			
weekend_planner_lookahead_days	21	WeekendPlanner.js	Days ahead to look for upcoming trips/events			
monthly_review_enabled	true	MonthlyReview.js	Generate monthly life review on 1st of each month			
pantry_enabled	false	Pantry.js	Enable purchase history + auto-restock tracking			
pantry_restock_days_ahead	7	Pantry.js	Days ahead to predict and flag restock items			
pantry_ema_alpha	0.3	Pantry.js	EMA learning rate for pantry consumption model (higher = adapts faster)			
google_tasks_enabled	true	WebApp.js	Fetch Google Tasks in dashboard and chat			
morning_routine_enabled	true	WebApp.js (status)	Show Morning Routine tab in dashboard			
takeouts_enabled	true	WebApp.js (status)	Show Takeouts tab in dashboard			
experiments_enabled	true	WebApp.js (status)	Show Experiments tab in dashboard			
wish_list_enabled	true	WebApp.js (status)	Show Wish List tab in dashboard			
wishlists_enabled	true	WebApp.js (status)	Show Christmas Wish Lists (People tab)			

Key	Default	Module	What it controls			
dates_default_lead_time	30	ImportantDates.js	Default lead days for important date flags			
dates_high_urgency_days	1	ImportantDates.js	Days-until threshold for High urgency important date flag			
dates_medium_urgency_days	7	ImportantDates.js	Days-until threshold for Medium urgency important date flag			
memory_log_enabled	true	Memory.js	Enable VERA memory/reflection log			
memory_snapshot_day	0	Memory.js	Day of week for memory snapshot (0=Sun)			
memory_log_retention_months	12	Memory.js	Months of memory log to retain before pruning			
monthly_disposable_income	5000	FinancialGoals.js	Monthly disposable income for financial goal scenario simulation			
pattern_max_flags	2	PatternRecognition.js	Max new pattern flags per nightly run			
pattern_dedup_days	7	PatternRecognition.js	Days to suppress same pattern key after it fires			
weather_location	blank	Weather.js	City name for weather ticker (e.g. `Austin, TX`)			

Key	Default	Module	What it controls			
travel_day_briefing_enabled	true	TravelDayBriefing.js	Enable travel day briefing messages			
travel_companions	blank	TravelDayBriefing.js	Default travel companions string			
summary_sheet:*	none	Summaries.js	External sheet hook: value = `SheetID\	TabName\	CellRef\	metric_name`

Script Properties Project Settings → Script Properties in Apps Script editor

Property Key	Required?	Used By	Description
CLAUDE_API_KEY	Required	Claude.js	Anthropic API key for claude-sonnet-4-6
VERA_SHEET_ID	Required	Code.js (CONFIG)	Google Sheet ID of the Life OS sheet
MORNING_NUDGE_EMAIL	Required	Code.js (CONFIG)	Email address for morning nudge and error alerts
VERA_WEB_TOKEN	Required	WebApp.js	Auth token for all WebApp GET/POST requests (`?token=...`)
VERA_LOGO_FILE_ID	Optional	Code.js, MonthlyReview.js	Google Drive file ID for VERA logo in emails
VERA_DASHBOARD_URL	Optional	Code.js, MonthlyReview.js	Netlify/dashboard URL — shown as button in emails
SLACK_BOT_TOKEN	Optional	Slack.js	Slack bot OAuth token (xoxb-...)
SLACK_CHAT_CHANNEL_ID	Optional	Slack.js	#vera-chat channel ID (bidirectional conversation)
SLACK_NOTIFICATIONS_CHANNEL_ID	Optional	Slack.js	#vera-notifications channel ID (flags with Block Kit buttons)
SLACK_LOGS_CHANNEL_ID	Optional	Slack.js	#vera-logs channel ID (nightly run summaries, errors)
SLACK_AHMED_USER_ID	Optional	Slack.js	Ahmed's Slack member ID for @mentions on High flags

Property Key	Required?	Used By	Description
SLACK_VICTORIA_USER_ID	Optional	Slack.js	Victoria's Slack member ID
SLACK_ALLOWED_USER_IDS	Optional	Slack.js	Comma-separated Slack member IDs authorized to chat with VERA
TRANSACTIONS_SHEET_ID	Optional	Finance.js, WebApp.js	Separate Google Sheet ID for transaction history (Empower CSV format)
SAT_SHEET_ID	Optional	Finance.js, WebApp.js	Simple Ass Tracker Google Sheet ID (household budget)
AVIATIONSTACK_KEY	Optional	FlightStatus.js	AviationStack API key for flight status monitoring
WEATHER_API_KEY	Optional	Weather.js, WebApp.js	OpenWeather API key for weather ticker and destination forecasts
VERA_SEARCH_API_KEY	Optional	Chat.js	Serper.dev or Tavily API key for web search tool in chat
VERA_SEARCH_ENGINE	Optional	Chat.js	`serper` (default) or `tavily`
PACING_MODE_ACTIVE	Runtime	Pacing.js	Set to `true` when pacing mode is active (written by VERA)
PACING_MODE_ENDS	Runtime	Pacing.js	ISO date string when pacing mode expires
VACATION_MODE_ACTIVE	Runtime	Pacing.js	Set to `true` when vacation mode is active
VACATION_MODE_ENDS	Runtime	Pacing.js	ISO date string when vacation mode expires
VACATION_TRIP_NAME	Runtime	Pacing.js	Trip name stored when vacation mode activates
PACING_FIRST_DETECTED	Runtime	Pacing.js	Epoch ms timestamp when overload was first detected
PACING_OFFER_FLAG_KEY	Runtime	Pacing.js	Flag key for the pacing offer flag (dedup guard)
FLIGHT_STATUS_CACHE	Runtime	FlightStatus.js	JSON cache of last known flight statuses
LAST_NIGHTLY_RUN	Runtime	Slack.js (App Home)	Timestamp of last successful nightly run
SHOPPING_LIST_DOC_ID	Optional	Shopping.js	Google Doc ID for shared shopping list

Property Key	Required?	Used By	Description
LIFE_PLAN_DOC_ID	Optional	FinancialGoals.js	Google Doc ID for Life Plan document
SLACK_PINNED_MORNING_TS	Runtime	Slack.js	Slack message timestamp of pinned morning summary

Sheet Tabs TABS constant in Code.js — all 80+ tabs

Constant	Tab Name	Purpose	Auto-managed?
`TABS.FLAGS`	Flags	All VERA-generated flags with urgency, status, escalation	Auto
`TABS.TASKS`	Tasks	Open tasks with due dates, recurring, flagged status	Manual
`TABS.METRICS`	Metrics	VERA self-monitoring counts (tasks, flags, calendar); wiped nightly	Auto
`TABS.SUMMARIES`	Summaries	External life intelligence feed — [AUTO] rows wiped/rewritten nightly	Hybrid
`TABS.CONFIG`	Config	Key/Value config rows driving all VERA behaviour	Manual
`TABS.PROJECTS`	Projects	Multi-step projects with Claude-generated subtasks	Manual
`TABS.GOALS`	Goals	Yearly goals Kanban (To Do / Doing / Done / Paused)	Manual
`TABS.PTO`	PTO	PTO planner snapshot — written nightly by writePTOSnapshot_	Auto
`TABS.PTO_MEMORY`	PTO Memory	Stateful PTO suggestion history; declined windows blacklist	Auto
`TABS.REMINDERS_MEMORY`	Reminders Memory	Reminder cooldown log — prevents duplicate nudges	Auto
`TABS.INTEREST_LEDGER`	Shared Interests	Shared interest ledger for Ahmed + Victoria	Manual
`TABS.BILLS`	Bills	Recurring bill tracker with paid status	Manual
`TABS.RECIPES`	Recipes	Recipe list with ingredients and tags	Manual

Constant	Tab Name	Purpose	Auto-managed?
TABS.MEAL_PLAN	Meal Plan	Weekly dinner planner — Saturday reset auto-archives + seeds next week	Hybrid
TABS.TAKEOUT_RESTAURANTS	Takeout Restaurants	Favorite takeout restaurant list	Manual
TABS.TAKEOUT_ITEMS	Takeout Items	Takeout menu items linked to restaurants	Manual
TABS.HOME_ITEMS	Home Items	Warranties + service log for home items	Manual
TABS.IDEAS	Ideas	Braindump repo — raw thoughts + structured ideas	Manual
TABS.ITINERARY	Itinerary	Trip itinerary items (flights, hotels, activities)	Manual
TABS.TRIP_META	TripMeta	Trip context and sentiment notes per trip key	Manual
TABS.PACKING_ITEMS	PackingItems	Per-trip packing list items by person and category	Manual
TABS.COUNTRIES	Countries	Countries visited tracker	Manual
TABS.BUCKET_LIST	Bucket List	Travel bucket list with priority stars	Manual
TABS.TRIP_RECOMMENDATIONS	TripRecommendations	AI-generated trip activity and dining suggestions	Auto
TABS.PROCESSED_EMAILS	Processed Emails	Email parser dedup + outcome log	Auto
TABS.MORNING_ROUTINE	Morning Routine	Daily routine checklist — nightly checkbox reset	Hybrid
TABS.GYM_LOG	Gym Log	Gym session attendance log	Auto
TABS.PURCHASE_HISTORY	Purchase History	Pantry purchase history + consumption intelligence	Manual
TABS.CAREER_POSITION	Career Position	Current role snapshot	Manual
TABS.CAREER_GOALS	Career Goals	Long-horizon career targets	Manual
TABS.CAREER_PROGRESSION	Career Progression	Career timeline / roles held	Manual
TABS.CAREER_DEVELOPMENT	Career Development	Skills, courses, focus areas	Manual
TABS.CAREER_WINS	Career Wins	Professional achievement log	Manual

Constant	Tab Name	Purpose	Auto-managed?
TABS.CAREER_NETWORK	Career Network	Professional relationships tracker	Manual
TABS.PRESCRIPTIONS	Prescriptions	Medication tracker for Ahmed + Victoria	Manual
TABS.CREDIT_CARDS	Credit Cards	Card metadata + ownership (11 cards)	Manual
TABS.CARD_REWARDS	Card Rewards	Per-card category reward rates	Manual
TABS.CARD_PERKS	Card Perks	Monthly/annual perk tracker with last-used date	Manual
TABS.LOYALTY_PROGRAMS	Loyalty Programs	Points/miles balances per program	Manual
TABS.REWARDS_GOALS	Rewards Goals	Redemption goal tracking	Manual
TABS.GIFT_PEOPLE	Gift People	People list for gift ideas (Ahmed + Victoria seeded)	Manual
TABS.GIFT_IDEAS	Gift Ideas	Individual gift ideas linked to people	Manual
TABS.IMPORTANT_DATES	Important Dates	Birthdays, anniversaries, meaningful dates — synced from calendar	Hybrid
TABS.CHORES	Chores	Household chore checklist — cadence-based nightly reset	Hybrid
TABS.TRAVELER_PROFILES	Traveler Profiles	Passport + traveler profiles for visa checking	Manual
TABS.CONTRACTS	Contracts	Active contract + agreement tracker	Manual
TABS.VEHICLES	Vehicles	Vehicle tracker — oil, service, registration, tires	Manual
TABS.FINANCIAL_GOALS	Financial Goals	Financial goals + what-if scenario planning	Manual
TABS.FINANCIAL_SCENARIOS	Financial Scenarios	Saved what-if scenarios per goal	Manual
TABS.EMAIL_FOLLOW_UPS	Email Follow-ups	Email Admin follow-up tracking	Auto
TABS.BOOKS	Books	Reading list per person	Manual
TABS.COURSES	Courses	Courses and learning content tracker	Manual

Constant	Tab Name	Purpose	Auto-managed?
TABS.SKILLS	Skills	Skill building + practice log	Manual
TABS.EXPERIMENTS	Experiments	Personal experiment tracker with hypothesis + outcomes	Manual
TABS.EXPERIMENT_CHECKINS	Experiment Checkins	Per-experiment check-in log	Manual
TABS.RESOURCES	Resources	Reference links + docs (Explore tab)	Manual
TABS.BUCKET_ACTIVITIES	BucketActivities	Per-destination activity list linked to bucket list	Manual
TABS.WISH_LIST	Wish List	Aspirational purchase tracker per person	Manual

Slack Channels Slack.js — getSlackChannelId_ channel map

Channel	Property Key	Direction	What posts there
#vera-chat	SLACK_CHAT_CHANNEL_ID	Bidirectional	Ahmed + Victoria conversational chat. Messages processed async via CacheService queue. @mentions resolve to names via SLACK_AHMED_USER_ID / SLACK_VICTORIA_USER_ID.
#vera-notifications	SLACK_NOTIFICATIONS_CHANNEL_ID	Outbound only	Flag alerts with Block Kit Acknowledge + Snooze buttons. High-urgency flags @mention Ahmed via SLACK_AHMED_USER_ID. Morning nudge summary. Weekend Planner.
#vera-logs	SLACK_LOGS_CHANNEL_ID	Outbound only	Nightly run summaries (flag counts, step warnings, elapsed time). WebApp errors. Flight status updates. Feature fire confirmations.

WebApp GET Actions doGet() in WebApp.js — all require ?token=VERA_WEB_TOKEN

action param	Additional params	Description
`status`	—	Flag counts (total, active, high/med/low), last run date, feature flags for dashboard tab visibility, travelConfig
`flags`	`filter=active` (opt)	All flags, or unacknowledged+unresolved only when filter=active
`tasks`	—	Open tasks from Tasks tab
`get_google_tasks`	—	Tasks from Google Tasks API (gated: google_tasks_enabled)
`complete_google_task`	`id`	Mark a Google Task as complete
`summaries`	—	Summaries tab data
`acknowledge`	`id`	Acknowledge a flag by ID
`snooze`	`id`, `days`	Snooze a flag for N days
`resolve`	`id`	Resolve a flag by ID
`complete_task`	`id`	Mark a task as Done
`add_task`	`task`, `dueDate`, `recurring`	Add a new task
`update_task`	`id`, `field`, `value`	Update a task field
`delete_task`	`id`	Delete a task by ID
`shopping`	—	Shopping list by store
`shopping_toggle`	store, item	Toggle shopping item purchased state
`shopping_add`	store, item	Add item to shopping list
`shopping_delete`	store, item	Delete shopping item
`shopping_update`	params	Update shopping item text
`projects`	—	All projects with subtasks
`complete_project_task`	`row`	Complete a project subtask
`add_project_task`	project, task, priority, dueDate	Add subtask to project
`update_project_task`	params	Update project subtask
`delete_project_task`	params	Delete project subtask
`goals`	—	Yearly goals
`add_goal / update_goal / delete_goal`	params	Goal CRUD

action param	Additional params	Description
interests	—	Shared Interest Ledger
pto	—	PTO snapshot + stats
budget	—	SAT budget data
bills	—	Bills list
bills_toggle	row	Toggle bill paid status
calendar_bills	—	Calendar-derived bill events
tx_list	—	Transaction list from Transactions sheet
recent_transactions	—	Recent transactions summary
cashflow	month, year	Monthly cashflow projection
recipes	—	Recipe list
meal_plan	week	Meal plan for a given week
set_meal / update_meal_status / suggest_meals	params	Meal plan management
takeouts	—	Takeout restaurants + items
homesteward	—	Home items + warranties + service log
ideas	—	Idea braindump + thought inbox
itinerary	tripKey	Trip itinerary items
get_trip_meta / set_trip_meta	tripKey	Trip context/sentiment notes
get_packing	tripKey	Packing list for a trip
generate_packing	tripKey, start, end	AI-generate full packing list
countries	—	Countries visited
get_bucket_list	—	Travel bucket list
flight_statuses	tripKey	Current flight statuses for a trip
force_flight_statuses	tripKey	Force-refresh flight statuses via AviationStack
recommendations	tripKey	AI trip activity/dining recommendations
chat	message, session	VERA conversational AI (Phase 4)
morning_routine	—	Morning routine checklist
gym_log	—	Gym session log

action param	Additional params	Description
career	—	Career data (position, goals, progression, wins, network)
prescriptions	—	Medication tracker
cards	—	Credit card hub (cards, rewards, perks, loyalty, goals)
get_gift_data	—	Gift people + gift ideas
get_important_dates	—	Important dates list
get_chores	—	Household chores with cadence + status
financial_goals	—	Financial goals + scenarios
get_vehicles	—	Vehicle tracker data
get_contracts	—	Active contracts
health_appointments	—	Health appointments derived from DR: calendar events
get_growth	—	Books, courses, skills (Growth / Explore tab)
get_experiments	—	Personal experiments + check-ins
get_resources	—	Reference links and docs
get_wish_list	—	Aspirational wish list
get_pacing_status	—	Current pacing + vacation mode status
get_visa_requirements	origin, destination	Visa requirements for a traveler profile
dest_weather	location	Weather forecast for destination
tx_aliases / set_tx_alias	params	Transaction merchant alias management

WebApp POST Actions doPost() in WebApp.js — JSON body required

action	Body params	What it does
chat	message, session	Process a chat message (same as GET chat)
acknowledge	id	Acknowledge a flag by ID
snooze	id, days	Snooze a flag for N days
resolve	id	Resolve a flag by ID

action	Body params	What it does
add_takeout_restaurant	name, cuisine, phone, website, rating, notes	Add a takeout restaurant
delete_takeout_restaurant	name	Delete a takeout restaurant
add_takeout_item	restaurant, item, description, rating, notes	Add a menu item to a restaurant
delete_takeout_item	restaurant, item	Delete a takeout menu item
log_purchase_run	items array	Log a grocery/shopping run to Purchase History
Note: Slack Events API POSTs (type: event_callback) and Slack form payloads (Block Kit interactions) are handled automatically without auth token — detected by body.type or e.parameter.payload.		

Cross-Domain Patterns PatternRecognition.js — all 7 patterns

Pattern Key	Trigger Conditions	Urgency
cross_high_stress_compound	overdueTasks ≥ 3 AND activeHighFlags ≥ 2 AND gymSessionsThisWeek = 0	High
cross_goal_behaviour_drift	activeGoals ≥ 2 AND gymSessionsThisWeek < (gymTarget × 0.5) AND overdueTaskCount ≥ 2	Medium
cross_social_calendar_gap	calendarEventsThisWeek ≤ 1 AND pacingMode = false AND vacationMode = false	Low
cross_overload_no_pacing	intensityLevel = 'high' AND pacingMode = false AND activeHighFlags ≥ 2 AND overdueTaskCount ≥ 3	Medium
cross_meal_chaos	takeoutRatio ≥ 0.70 (≥70% of meals are takeout/eating out) AND overdueTaskCount ≥ 2	Low
cross_backlog_accumulation	neglectedTaskCount (age ≥ 14 days) ≥ 5 AND calendarEventsThisWeek ≥ 3	Medium
cross_health_neglect	overdueHealthAppts ≥ 2 AND gymSessionsThisWeek = 0	Medium

Health Appointment Default Intervals HEALTH_APPT_INTERVAL_DEFAULTS_ in HealthTracker.js

Type (exact match keys)	Interval (months)
`physical` / `annual physical`	12
`dental` / `dentist` / `dental cleaning` / `teeth cleaning`	6
`eye exam` / `optometrist` / `ophthalmologist`	12
`dermatology` / `dermatologist`	12
`gynecology` / `obgyn` / `gynaecology`	12
`therapy` / `therapist`	1
`cardiology` / `cardiologist`	12
`endocrinology` / `endocrinologist`	12
`allergist` / `allergy`	12
`chiropractor` / `chiropractic`	1
`urgent care`	0 — never flagged (episodic)
Any other type	12 (fallback)